

Make your own game with AI.

You pay for ChatGPT or Claude. Here's how to get a real game out of it: first a complete shooter in one chat, then a much better one with a coding agent. You don't write code. You play, and you decide.

By **Mudit** · @ai_with_mudit · November 2026

START HERE

One game that grows.

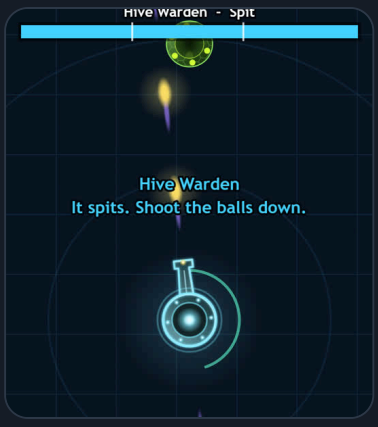
You build one shooter called **Hold the Middle**. You stand in the centre, enemies come from all sides, and you point and shoot.

SMALL VERSION



Built in one chat. One file. Plain shapes, real sound, waves, a score.

MEDIUM VERSION



Built with a coding agent. Upgrades, three bosses, its own art and music.

Along the way you learn the method, and which tool fits which size of game. Then you use it on **your own idea**.

TESTED BEFORE YOU GOT IT

AI agents built both versions from the prompts in this guide. On the medium one, a separate agent tested every milestone, and what it found went back into the prompts, so these are the fixed versions, not first drafts. Then I played the finished game myself.

ABOUT MY CRICKET GAME

My cricket game is a much bigger project, built in Unity. Its code comes when it's finished. The guide to building with Unity and Claude Code comes when the game is fully built. Everything here runs in a web browser.

01 · PICK YOUR SIZE

The size of the game picks the tool.

	WHAT IT IS	BUILD IT WITH
Small	One core action on one screen. A few enemies or objects. A score. A sitting or two.	ChatGPT or Claude, in the browser
Medium	Several systems working together: levels, upgrades, bosses, real art and sound, saved progress. Days.	Claude Code or Codex
Big	3D worlds, online multiplayer, long stories, huge amounts of content. Months, or a team.	A real engine, like Unity

A quick test for your idea

- **Say it in one sentence.** "You stand in the middle and shoot what's coming." If you need three sentences, it isn't small.
- **Count the things that must work together.** One or two is small. Three or four, start small and see. Five or more is medium.
- **Does it need other people online, or a 3D world?** Then it's big. Find the fun part, and build that as medium first. Page 28 does this for you.

WHY START SMALL, EVEN IF YOU WANT MEDIUM

You find out if the idea is fun within one sitting. And your small game becomes the exact starting point for the medium one, as it does in this guide.

02 · WHAT YOU NEED

What each route takes.

	SMALL	MEDIUM
AI	ChatGPT or Claude, on a paid plan	Claude Code or Codex, on a paid plan
Computer	Any laptop with a browser	A laptop you can install things on
Also	Notepad or TextEdit	Node.js and Git, both free. Your agent can walk you through them.
You'll get	One file, to play and share	A full project with tests, and a build to put online

Be ready for the real cost

This guide is free. Making the game costs you a subscription and your time, and that's normal. Good results take one or the other.

- **Limits:** every plan has usage limits. Long agent sessions use them fast. If you hit one, your files are safe, so come back when it resets. A bigger plan means fewer breaks.
- **Time:** by my estimate, agents ran for about 7 hours in total on my test build's medium milestones, including testing. Plan it across several days.
- **Your part:** you play it and decide. That's the job no agent can do for you.

THE PROMPTS AND THE SMALL GAME ARE ONLINE

Copying long prompts from a PDF is painful. Every prompt here is also at **mudit.si/make-your-own-game-prompts**. Already use Claude Code or Codex? Skip to page 15, and start from the small game at **mudit.si/hold-the-middle-small**. Save it as `game.html`.

03 · SMALL ROUTE

How it works in a chat.

1. **Paste a prompt.** Start a new chat. Paste the rules on the next page and Step 1 together, as one message.
2. **Copy all the code** from the reply, using the copy button on the code block.
3. **Save it.** Paste it into Notepad, and choose Save As. Name it `game.html`, and set the type to **All files**. On a Mac, use TextEdit in plain text mode.
4. **Double-click the file.** It opens in your browser, and that's your game.
5. **Play it, then paste the next step** into the same chat.

KEEP EVERY VERSION

After each step that works, save a copy: `game-v1.html`, `game-v2.html`, and so on. Those files are your undo button. If a later step ruins something, you open the last good one.

BLANK PAGE, OR THE CODE GOT CUT OFF?

Page 14 has the exact words to send.

IF YOUR CHAT SHOWS A PREVIEW

Many chats can show the game right there: artifacts in Claude (if you don't see them, turn on code execution and file creation in Settings), and canvas in ChatGPT. Use that as a shortcut, but save the file anyway. A file always works, and it's what you'll share.

03 · SMALL ROUTE

The rules. Paste these first.

These rules stop the bugs AI-written browser games usually have: sound that won't start, speed that changes by screen, blurry graphics, broken touch.

```
You are helping me make a small game that runs in a web
browser. I can't code, so I will only save your file and
play it. Follow these rules for this whole chat:

1. Everything goes in one HTML file. No libraries,
images, sound files or fonts, and nothing loaded from any
other website.
2. Every time you change the game, give me the whole
file, from the first line to the last, in one code block.
Never give me a part of it, and never write things like
"the rest stays the same".
3. Change only what I ask for. Everything else must keep
working exactly as it does now.
4. Put the numbers that most change how the game plays in
one block called SETTINGS at the very top of the script,
each with a plain-English comment.
5. All movement and timers use real elapsed time, so the
game runs at the same speed on every screen. If the tab
was hidden, the game must not jump forward when I come
back.
6. The game fills the whole window, looks sharp on any
screen (use devicePixelRatio), and works on a wide laptop
window and a tall phone screen. It adapts when the window
changes size.
7. Use pointer events, so a mouse and a finger work the
same way. The page must not scroll, zoom or select text
when I touch the game.
8. Make all sound in code with the Web Audio API. Start
it on my first click or tap, because browsers block sound
before that. If sound fails, the game must keep working.
9. After the code, write one short line saying what
changed and one thing I should test. Don't explain the
code.
```

Paste them once, together with Step 1. You only need them again if you start a new chat.

SMALL ROUTE · STEP 1 OF 6

1 Aim and shoot.

The very first thing you build is the action the player repeats a thousand times. It already has sound, because a silent action can't be judged.

```
Step 1: aim and shoot.

Make a dark screen with my player, a bright circle, in
the exact centre. A short barrel on the circle always
points at my mouse or finger.

While I hold the mouse button or keep my finger down, the
player fires small bullets towards that point at a steady
rate. Each shot makes a short, sharp sound. Bullets that
leave the screen are removed.

Don't add enemies, score or anything else yet.

Done means: I can aim anywhere and hold to keep firing,
and I hear every shot.
```

YOU SHOULD SEE

A circle in the middle that fires wherever you point. Click once if you don't hear it: browsers hold sound until you do.

TRY TO BREAK IT

Click very fast, then resize the window. Does the player stay in the centre?

2 Enemies and hits.

```
Step 2: enemies and hits.

Add enemies: red squares that appear just outside a
random edge of the screen and move straight towards the
player at a steady speed. Start them slow: each one takes
about 6 seconds to reach the player, on any screen size.
A new one appears at a steady rate.

When a bullet touches an enemy, both disappear. Show a
small burst of bright dots there and play a short hit
sound, different from the shot sound.

When an enemy reaches the player, it just stops there for
now. Don't add score or losing yet.

Keep the aiming and shooting exactly as they are.

Done means: enemies come from all four sides, and every
hit gives me a burst and a sound.
```

YOU SHOULD SEE

Enemies coming in from every side, and vanishing with a burst and a sound when you hit them.

TRY TO BREAK IT

Stop shooting, and let lots of them pile up. Does it slow down or crash?

3 Score, losing, restart.

Now it's a game: you can lose. Name it in the prompt: Hold the Middle, or a name of your own.

```
Step 3: score, losing and restart.

Before the game starts, show the name [your game's name]
and "Click or tap to start". Nothing moves until then.

Add a score at the top of the screen: 1 point per enemy
hit. If an enemy touches the player, the game stops and I
see "Game over", my score, my best score since I opened
the page, and "Click or tap to play again".

Ignore clicks and taps for the first half second on the
game over screen, so I can't restart by accident while
still shooting.

Restarting must reset everything: no enemies or bullets
left over, score back to zero, starting speed again.

Keep the aiming, shooting, enemies and hit effects
exactly as they are.

Done means: I can lose, see my score and play again, as
many times as I like.
```

YOU SHOULD SEE

A start screen, a score, a game over screen, and a clean restart.

TRY TO BREAK IT

Restart ten times quickly. Does anything carry over or speed up?

4 Make it feel good.

The rules don't change at all in this step. This is the step that shows you what game feel is.

```
Step 4: make it feel good.

The rules don't change. Only how it feels changes. Add
these effects:

- Each shot: the player kicks back a little and a small
flash appears at the barrel.
- Each hit: the enemy flashes white for a moment before
it bursts, the screen shakes a tiny bit, and the hit
sound changes pitch slightly at random.
- Game over: a bigger shake and a low sound.

Keep the shake small, so I can always see every enemy.
Effects must never hide an enemy.

Put a setting called EFFECTS_ON at the top of SETTINGS.
When I change it to false, every effect in this step
turns off, and the game plays exactly as it did before.

Keep the rules, score and restart exactly as they are.

Done means: the same game feels much more satisfying, and
switching EFFECTS_ON shows me the difference.
```

DO THIS YOURSELF

Right-click `game.html`, choose Open with, then Notepad (or TextEdit on a Mac). Find `EFFECTS_ON`, change `true` to `false`, save, and refresh the game. Play for a minute. Change it back. It's the same game, and it feels completely different.

TRY TO BREAK IT

When lots is exploding, can you still see every enemy?

5 Waves.

```
Step 5: waves.

Replace the steady flow of enemies with waves. Show "Wave
1" in large text for a moment, send the wave, and when
all its enemies are gone, give me a two second break and
start the next one.

Each wave has more enemies, and they move a little
faster. From wave 3, add a second enemy: a small orange
triangle that is much faster but appears less often. It
behaves like the red squares in every other way.

Show the current wave number next to the score, and on
the game over screen.

Put the settings for how quickly waves get harder in
SETTINGS, with a clear comment on each. Speed must have a
maximum, so a very long game stays fair.

Keep the aiming, shooting, effects, score and restart
exactly as they are. Restarting begins again at wave 1.

Done means: it starts easy, gets harder, and I can change
how hard it gets by editing numbers at the top.
```

DO THIS YOURSELF

Open the file in Notepad, find `SETTINGS`, and read the comments. Change one number, save, and refresh. That's you tuning your own game.

TRY TO BREAK IT

Set a number far too high. If it goes wrong, open your last saved version.

SMALL ROUTE · STEP 6 OF 6

6 Phone ready.

```
Step 6: make it phone ready.

No new features. Check and fix these:

- On a tall phone screen, the score, wave text and game
over text all fit and are easy to read. The player stays
in the exact centre.
- The enemies' starting distance and speed feel fair on a
tall narrow screen as well as a wide one.
- Holding a finger down doesn't open a menu, select text,
or scroll the page.
- Turning the phone or resizing the window doesn't break
anything.
- If I switch to another tab or app, the game pauses, and
it continues when I come back.
- Sound still works after coming back. Add this line,
which may let sound play in silent mode on iPhones (other
browsers ignore it): if (navigator.audioSession)
navigator.audioSession.type = "playback";

Keep everything else exactly as it is.

Done means: it plays well with one thumb on a real phone.
```

YOU SHOULD SEE

Nothing new on your laptop. The real test comes when you open your link on a phone (page 30).

TRY TO BREAK IT ON YOUR PHONE

Silent mode, turning the phone, switching apps and coming back.

THAT'S A COMPLETE GAME

You can share it now. Page 30 shows how. Or keep going, and turn it into something much bigger.

04 · THE METHOD

Five rules that do most of the work.

1. **One thing at a time.** One request adds one system. Big vague asks are what break.
2. **You judge by playing.** Not by reading code, and not by what the AI says. If it says "done" and it feels wrong, it's wrong.
3. **Feedback comes with the action.** Every action gets its sound and its visual in the same step, not later. You can't judge a flat, silent action.
4. **Save what works.** A version you can go back to is what lets you try things without fear. With an agent, that's a commit.
5. **Stop after two failed fixes.** Don't ask for a third. Go back to your last good version (with an agent, say "undo everything since the last commit"), and ask for the change in a smaller piece.

Why you stay in charge: my cricket game

Perfect lofted shots went for six 98% of the time. So the fielders I'd just added never got tested. I spotted that in the numbers, and we made lofts riskier.

I kept getting run out. The cancel-the-run key was F. My note was: "in that panic situation, the key 'f' is difficult to press there is too little time". We moved it to the down arrow. You only learn that with your hands on the keys.

04 · THE METHOD

When something goes wrong.

Report a bug like this

```
When I [what I did], I expected [what should happen], but
[what happened]. Fix only this, and give me the whole
file.
```

HARD TO FIX

"The enemies are broken."

EASY TO FIX

"When I restart, I expected no enemies, but the old ones are still there."

With an agent, use the version on page 18.

The code stopped halfway

```
Your code stopped before the end of the file. Give me the
whole file again, from the first line to the last, in one
code block.
```

The chat got slow or forgetful

Start a new chat. Paste the rules, then this, then your whole file. It also works in a different AI tool.

```
Here is my game so far. It works. Don't change anything
yet. Reply only "Ready", and wait for my next step.
```

05 · THE WALL

When chat has run out.

Chat takes you a long way, and then it starts fighting you. These are the signs:

- Every change rewrites a file so long that replies get cut off or take minutes.
- Fixing one thing breaks something else, again and again.
- The AI forgets decisions you made earlier.
- You want real art, several levels, or saved progress, and one file can't hold it comfortably.

What a coding agent changes

	CHAT	CODING AGENT
Your game	One file you copy and save	A real project on your laptop
Running it	You do	It runs and tests it too
Memory	This chat only	A rules file it reads every time
Undo	Your saved copies	Git, a full history
Checks	You	Tests it writes, plus you

IF YOU'RE PAYING FOR CHATGPT OR CLAUDE, YOU MAY ALREADY HAVE ONE

Claude Code comes with paid Claude plans, in the Claude desktop app's Code tab. **Codex** comes with ChatGPT plans, in its desktop app. If you've never opened it, this is the part of your subscription you aren't using. No terminal needed.

06 · MEDIUM ROUTE

Set up the project.

1. **Open your agent.** Claude Code: the Claude desktop app, Code tab. Codex: the Codex app.
2. **Make a folder** for your game, and put your `game.html` in it.
3. **Choose that folder** in the app. Keep the setting that asks you before it changes things, and read each one-line request.
4. **Check your tools.** Ask: "Do I have the LTS version of Node.js, 22.12 or newer, and Git? If not, tell me exactly how to install them."
5. **Paste Milestone 1,** with the rules file from the next page inside it.

Milestone 1: set up

```
This folder has my game, game.html, built in a chat. I want to
turn it into a real project. Don't change the game yet.

1. Read game.html, and tell me in five lines what the game
does and how it's built.
2. Set up git.
3. Create the rules file, with exactly this text:
[paste the rules file here]
4. Commit game.html and the rules file as the first commit.
5. Tell me the three things most likely to go wrong when we
rebuild it properly.

Stop there.

Done means: there's one commit, the rules file exists, and I
understand your summary.
```

I ran this build in Claude Code. Codex follows the same method, but this exact build hasn't been run in it.

EACH MILESTONE GOES THE SAME WAY

The agent shows a plan, you say yes, it builds, and it tells you what to try. You play, a second agent tests (page 24), and then you say **"commit this"**. The agent does it, and that point becomes your undo. To go back, say **"undo everything since the last commit"**.

06 · MEDIUM ROUTE

The rules file.

It's saved as `CLAUDE.md` for Claude Code, or `AGENTS.md` for Codex. The agent reads it every time. It's why the agent stays on track.

```
# Project rules

A 2D browser game. Phaser 4.2.1, TypeScript, Vite. I don't read code: I
play the game and judge it.

## How we work
- One system per request. Show me a short plan first, and wait for my
yes.
- Game rules live in plain TypeScript in src/rules, with no Phaser
imports, and have Vitest tests.
- Numbers and content live in data files in src/data, so I can change
them without touching the game.
- Don't tell me something works until the tests pass, the build passes,
and you've run the game in a browser test. Then start the game for me
and give me the link, or give me the exact command and where to type
it, and tell me what to try.
- After a fix fails twice, stop and tell me. Don't try a third.
- Don't add features I didn't ask for. Don't install anything new
without telling me why first.
- Work only inside this folder.

## Git
- Commit only when I say "commit this". Each commit is my undo button.
- If I say "undo everything since the last commit", first tell me what
I'd lose, then do it.
- Never force-push or rewrite history.

## Phaser 4, not Phaser 3
- Before using a Phaser feature, read the matching guide in
node_modules/phaser/skills.
- Most tutorials online are Phaser 3 and will mislead you. setTintFill,
Light2D and BitmapMask are gone.
- Math.TAU is now a full circle (2 pi). It used to be pi/2. Use radians
carefully in aiming code.

## Game feel
- Every player action gets feedback in the same change: sound,
movement, a visual.
- Effects never hide an enemy or a threat. One setting, EFFECTS_ON,
turns them all off.

## Art and sound
- Only use assets whose licence allows it. Record each one's source and
licence in CREDITS.md.

## Keep my laptop safe
- Browser tests use Playwright's bundled browser only. Never launch my
installed Chrome or Edge, and never touch my browser profile.
- Delete temporary browser profiles and test folders on every exit,
including failures and crashes.
- Run any new repeated or long test once first, and tell me before
repeating it.
```

Phaser is a free framework for browser games.

2 Rebuild it properly.

It looks the same at the end, and that's the point. Now it has a real structure, with tests.

```
Rebuild this game as a real project. Show me your plan
first, and wait for my yes.

Use Phaser 4.2.1, TypeScript and Vite. It must play the
same as game.html: same controls, enemies, waves, score,
game over, effects and sounds. Don't add anything new.

- Rules such as scoring, waves and enemy speed go in
src/rules, with tests.
- Every number in the old SETTINGS block goes in
src/data/settings.ts, with a plain-English comment.
- Add a browser test with Playwright's bundled browser
that starts the game, holds to fire, and checks that the
score can rise and the game can end.
- It must look at least as sharp as game.html on a high-
resolution phone. Phaser doesn't do this by default, so
handle the device pixel ratio yourself and keep aiming
accurate.
- Keep game.html as it is, so I can compare.

Done means: you start the game for me and give me the
link, or tell me the exact command and where to type it.
It plays like the old one, and the tests and build pass.
```

WHAT WENT WRONG IN MY BUILD

The first rebuild was blurry on sharp phone screens, and every automatic test passed anyway. The second agent failed the milestone over it, so that line is now in the prompt.

WHEN SOMETHING'S OFF, TELL THE AGENT

```
When I [what I did], I expected [what should happen], but
[what happened]. Find the cause first, then fix only
this, and check it the way the rules file says.
```

3 More enemies, as data.

This is the step that makes the project feel different from a chat. Enemies become a list you can edit yourself.

```
Add three new enemy types, and make enemies data, so I
can add more without touching the game's code.

- Zigzag: weaves from side to side as it comes in.
- Tank: slow and big, takes 3 hits, and shows damage on
each hit.
- Splitter: breaks into two small fast enemies when
killed.

Each enemy is one entry in src/data/enemies.ts: its look,
speed, health, movement pattern, score, and the wave it
first appears in. Each type must be easy to tell apart at
a glance and appear at a fair moment. Every enemy must be
visible for at least 1.5 seconds before it can touch me,
on any screen shape, including a tall narrow phone.

Keep the aiming, shooting, waves, score and effects as
they are.

Done means: all the enemy types show up over the waves,
and I can make a new enemy by copying one entry and
changing its numbers, with no other code change. Test
that yourself by adding one, then removing it.
```

DO THIS YOURSELF

Open `src/data/enemies.ts`, copy one entry, change its name, colour and speed, save, and play. You just designed an enemy without touching the game's code. In my test build, a new enemy worked with that one change.

4 Upgrades.

```
After each wave, pause the game and offer me a choice of
3 random upgrades. I pick one, and it lasts until the run
ends.

Start with 8 upgrades in src/data/upgrades.ts, such as
faster fire, double shot, piercing bullets, bigger
bullets, a shield that blocks one hit per wave, and a
slow-motion pulse. Each one must visibly change how the
game plays. Upgrades can stack, with sensible limits.

Show my chosen upgrades on screen during play. Restarting
clears them. Make sure no pick can make the game
impossible or trivially easy. Test this with a bot that
plays like a person, with reaction delay and aim error,
not with perfect aim, and adjust the numbers until it
shows that.

Keep the enemies, waves, score and effects as they are.

Done means: every run feels a bit different, because of
what I picked, and the choice screen works with a mouse
and a finger.
```



BOTS THAT PLAY LIKE PEOPLE

A perfect-aim bot never dies, so it tells you nothing. A bot with slow reactions and bad aim can actually test balance. Your hands still have the final say.

The screen is from my test build. Yours will look different, and it should.

5 Bosses and arenas.

Now a run has a real ending.

```
Add three arenas and a boss at the end of each one, so a
full run has a real ending.

- Each arena lasts 5 waves and has its own look:
background and colour mood.
- Each boss has a health bar and at least three phases
that clearly change how it attacks. The player stays in
the centre, so every attack must be something I can shoot
down, block or react to. Every attack gets a clear
warning before it lands.
- Boss 2 and 3 must feel different from boss 1, not just
tougher.
- Beating boss 3 shows a real win screen with my score
and time.

Bosses and arenas go in data files. Keep the aiming,
enemies and upgrades as they are.

Balance the full run with bots that play like people,
with reaction delay and aim error, at beginner and
average skill. An average player with sensible upgrades
should have a real chance of winning but not win easily,
and a beginner should reach the first boss. Difficulty
should rise through a boss's phases, not jump at one, and
changing one number shouldn't swing the win rate wildly.
Report those numbers.

Done means: I can play a full run from arena 1 to the win
screen, and no boss kills me without warning. Give me a
test setting to start at any arena, so bosses can be
checked quickly. It must only work while developing,
never in the published game.
```

WHAT WENT WRONG IN MY BUILD

The test start setting also worked in the published game, so anyone could jump to the final boss with free upgrades. And changing one number on the last boss swung win rates between 25% and 64%. Both were caught and fixed, and both are in this prompt now.

6 Art, sound and feel.

```
Make this look and sound like a finished game. No new
rules.

- Replace the shapes with a matching set of sprites for
the player, enemies, bosses and bullets, all in one
designed style. Draw them in code, or use a CC0 pack I've
downloaded into this folder (such as one from Kenney).
Record each source in CREDITS.md.
- Real sound effects for every action, and music that
suits each arena, generated in code or from a CC0 pack
I've downloaded. Use a volume setting, and start sound on
the first tap.
- Add feel using Phaser's built-in tools: particles on
hits, a soft glow on bullets, a brief freeze on big hits,
screen shake, and a boss death that feels huge.
- Add a proper title screen that says the goal in one
line, plus pause, settings (sound, music, EFFECTS_ON) and
a credits screen.
- Make beating a boss a real moment: bigger and longer,
with a beat before it ends. Make the win screen a
celebration, in the same style as the other screens.
- Check that no sprite is clipped by its own texture, and
look at real screenshots at phone and laptop size until
it reads as one designed game.

Everything stays readable: effects never hide an enemy, a
warning or the score. They calm down when the screen is
busy, and turning EFFECTS_ON off still leaves a clear
game.

Done means: someone seeing a screenshot wouldn't call it
a prototype, and it plays as smoothly as before.
```

TWO HONEST THINGS

Art: agents can't always download packs, because many sites block them. Download a pack in your own browser, or have it draw everything in code, as my test build did. **Sound:** no agent can hear. You have to judge this one yourself, so listen closely.

7 Finish and publish.

```
Finish the game, and get it ready to share.

- Save progress on this device: best score, furthest
arena, settings. If saving isn't available, the game must
still work.
- Phone: holding a finger doesn't scroll or open a menu,
text is readable, it handles turning the phone, and it
pauses when I leave the app. Measure the frame rate at
the busiest moment with the CPU slowed down 4 times, and
cut effects automatically if it struggles, never
warnings.
- Check the first load is fast, and show a loading bar.
- Build it for hosting as static files, and tell me
exactly how to put it online.
- Go through the whole game once more and list anything
that still feels unfinished, most important first.

Done means: I can open it from a link on my phone, play a
full run, close it, and find my progress still there.
```



PLAY IT ON A REAL PHONE

My test build passed everything on simulated phones. Only a real one tells you how it feels. That goes for your game too.

The win screen in my test build.

07 · THE SECOND AGENT

Never let the builder grade itself.

The most useful thing I learned on this build. An agent that just built something believes it works. A fresh one with no stake in it finds what's wrong. Start a **new chat on the same folder**, paste this, and don't tell it what you think.

```
You didn't build this, and you have no stake in it passing.
Find out whether this milestone is really done. A false pass
is the worst outcome.

The request was:
[paste the milestone prompt here]

1. Read the rules file, and check its rules were followed.
2. Run the tests and the build yourself. Don't take anyone's
word for it.
3. Run the real game and play it with a bot, at laptop size
and at phone size with touch, using Playwright's bundled
browser only. Aim at enemies, die, restart. Watch for console
errors.
4. Try to break it: rapid taps, restart many times, resize,
long play.
5. Look at screenshots yourself. Is anything clipped,
unreadable, overlapping, ugly, or not what the request asked
for?

Don't change anything. Report PASS or FAIL, one line of
evidence per item, bugs with exact steps to repeat them, and
what you couldn't check. You can't hear sound or feel the
game, so say so.
```

07 · THE SECOND AGENT

Fix what it finds, then test again.

Take the report back to your building chat, and paste this with it.

```
An independent tester checked this. Here is its report:
[paste the report here]

Fix only what failed. Don't change anything else. Find
the cause first, and tell me if you think the tester is
wrong about something, and why.
```

Then test again in another new chat. Agents can be wrong too, so read why, and trust your own play over any report.

What testers caught in my build, after the builder's own tests passed

- Blurry graphics on sharp phone screens
- Fast enemies crossing a phone screen in under a second
- A published game anyone could skip to the final boss in
- White blobs covering the score, and a win screen that looked unfinished

KNOW ITS LIMITS

A tester can't hear sound or feel the game. It also makes mistakes: in my build, one tester's numbers came from a problem in the test setup, not the game. It's a second pair of eyes, not the judge. You are.

More on this in my guide **The Second Agent Check**.

08 · KEEP YOUR LAPTOP SAFE

My own tests locked me out of my laptop.

Headless test runs of my installed Chrome made 298 failed Windows sign-ins in 3 days and locked me out of my own account. Leftover test profiles had also filled 7.7 GB of my drive.

An agent that's testing a game launches a browser again and again. So your rules file has the first four of these, and now you know why:

- ✓ **Tests use a bundled test browser only.** Playwright's, never your own Chrome or Edge, and never your browser profile.
- ✓ **Clean up every time.** Temporary profiles and folders get deleted on every exit, including failures and crashes.
- ✓ **New long-running tests run once first.** Check nothing odd happened before repeating them.
- ✓ **Stay in the folder.** Don't let an agent work outside your game's folder.
- ✓ **Read permission requests.** That one's on you. One line, every time.

IF SOMETHING ODD HAPPENS

Failed sign-ins, a full disk or a frozen laptop: stop all test runs first, and find out why before running anything again.

09 · ART, SOUND, COPYING

Copy an idea. Never a look.

Where art and sound come from

- **Drawn in code.** Your agent can do more than you'd expect. Hold the Middle's art, sound and music are all generated this way.
- **Kenney (kenney.nl).** Game art and sound under CC0, so you can use it freely, and credit is optional.
- **OpenGameArt.** Check every file's licence. CC-BY means you must credit the creator.
- Keep a `CREDITS.md` with each source and its licence.

What you can and can't copy

FINE

- ✓ A genre or a mechanic: shooters, catching, sliding puzzles
- ✓ Learning by rebuilding a classic, for yourself

DON'T

- ✗ A game's art, characters, music or name
- ✗ Real logos and team kits
- ✗ Saying or implying it's official

Real teams and players are a grey area, and the law isn't settled. I haven't shared my own game that uses real player names, for that reason. Use invented ones if you plan to publish.

General guidance, not legal advice. Sites have their own rules too: itch.io doesn't allow copying another creator's look, feel or functionality without permission.

10 · YOUR OWN IDEA

Now make yours.

Paste this into a new chat, with your idea in it. Small idea: paste the rules from page 6 first, then its steps one at a time. Medium idea: it gives you your own rules file and first prompt.

```
I can't code. I want to make a small browser game with AI, one step
at a time.

My game idea: [your idea in a sentence or two]

Help me in this order:

1. The core action: tell me the one thing the player does again and
again.
2. The right size. Tell me which fits, and why:
   - Small: one HTML file, built in a chat. A few hours.
   - Medium: a real project, built with a coding agent like Claude
Code or Codex. Several systems, real art and sound. Days.
   - Big: needs a real engine or a team, like a 3D open world or
online multiplayer. If my idea is big, tell me plainly, find the
fun part, and cut it down to a medium version.
   Tell me plainly what you cut and why.
3. The steps: write 6 steps for a small game or up to 10 for a
medium one, each adding one thing, and each leaving a game I can
play:
   - Step 1 is the core action, with a sound.
   - The next steps add what the player acts on, then how to win or
lose, with a restart.
   - One step is only for making it feel good: small effects and
sounds, with a setting at the top that turns them off.
   - One step is for getting harder over time.
   - The last step is finishing it: phone ready, and ready to
share.
   - Prefer a mouse or a finger at every step. If my idea needs a
keyboard by nature, keep the keyboard and tell me if that makes it
laptop-only.
   - If my game has levels or puzzles, each one must be possible to
finish. Tell me how to check that.
   - If exact timing is the heart of the game, like a rhythm game,
include a step to calibrate timing, and tell me how I can judge it.
4. Write each step as a prompt I can paste, in this form: what to
add, what must stay exactly as it is, and "Done means:" followed by
what I'll see when I play.
5. If it's medium, also give me: the best framework for this game
and why (check its current version, don't rely on memory, and tell
me if you can't check), a rules file for this project (how we work,
git, the framework's known traps, game feel, licences for art and
sound, and keeping my laptop safe: browser tests use only a bundled
test browser, never my own browser or profile, and clean up after
themselves), and a first prompt that sets up the project.

Don't write any game code yet.
```

10 · YOUR OWN IDEA

How to know it's too big.

- ✕ **It needs other people online.** Servers, syncing and cheating are a project of their own.
- ✕ **It has accounts or logins.** Now you're handling people's data.
- ✕ **It needs a big world.** Maps and content pile up fast.
- ✕ **The fun comes after an hour of play.** The first minute should already be fun.
- ✕ **You can't say what the player does again and again.**
- ✕ **You're designing a menu or a shop** before the core action feels good.

TESTED ON

Earlier versions of the prompt got a samosa-catching game, a calm slide puzzle with no way to lose, a rhythm game, a two-player snake game, and "GTA in Mumbai with online multiplayer". That last one got a plain answer on why it can't be built this way, then got cut down to the fun part: dodging traffic on one road.

What makes a first game work

- ✓ **Finish it.** A finished small game teaches you more than half of a big one.
- ✓ **Watch someone else play.** Say nothing. Where they get stuck is your next fix.

11 · SHARE IT

Get a real link.

Small game: itch.io

1. Make a free itch.io account, and create a new project.
2. Set the kind of project to **HTML**, and upload your file. Name it `index.html`.
3. Mark the file to be played in the browser, and tick **Mobile Friendly**.
4. Fill in the **AI disclosure** on the project's edit page, and mention it in your description. itch.io asks for this.
5. Publish, and copy the link. It works straight away, even if the listing in itch.io's browse pages takes days.

Medium game

Milestone 7 gives you exact steps for your own build. The agent makes a folder called `dist`, and static hosts like Netlify, Cloudflare Pages or itch.io (as a zip) serve it. Ask: "Which one fits this game, and what do I click?"

SHARING FROM INSTAGRAM

Links often open inside Instagram's own browser. If sound or the screen misbehaves, tap the **...** menu and choose the option to open it in your browser (the wording varies). Say so next to your link, and test on your own phone first.

SHOW ME

Built something from this? Send it to me at **@ai_with_mudit**. I want to play it.

12 · GOING BIG

When you outgrow the browser.

3D, real physics and big worlds need a real engine. That's what my cricket game uses: Unity, with Claude Code. The method doesn't change: one system at a time, you judge by playing. The setup is just bigger.

That guide comes when my cricket game is fully built, so it's based on the whole journey. Follow @ai_with_mudit to see it being built.

Cheat sheet

Pick	Small: chat. Medium: Claude Code or Codex. Big: an engine.
Build	One system per request. Sound and visuals in the same step.
Judge	Play it. Try to break it. Your hands decide.
Test	A second agent that didn't build it.
Save	A version or commit after every step that works.
Stuck	"When I... I expected... but..." Two failed fixes, go back.

Sources (checked November 2026)

Claude Code desktop: code.claude.com/docs/en/desktop-quickstart · Claude plans: claude.com/pricing
Codex: learn.chatgpt.com/docs/pricing
Phaser 4: phaser.io · github.com/phaserjs/phaser (MIT)
itch.io HTML games: itch.io/docs/creators/html5 · rules: itch.io/docs/creators/quality-guidelines
Kenney: kenney.nl/support · CC-BY: creativecommons.org/licenses/by/4.0
Web Audio, touch and audio sessions: developer.mozilla.org
Plans and menus change often. Not affiliated with Anthropic, OpenAI, Phaser or itch.io.